INFRASTRUCTURE / CONFIGURATION MANAGEMENT

Infrastructure Configuration Lab

One disposable MyScoutee-shaped edge. Two configuration tools. KVM or VMware as the VM provider.

Document control	Value
Document ID / version	MSC-INF-001 / 1.3.0 / EN
Repository	myscoutee-infra
Audience	Developers and operators learning host configuration management
Scope	Local Ubuntu 22.04 VM, nginx, Ansible and Puppet
Data and connectivity	Synthetic page/status; no MyScoutee backend or user data
Integration	Separate sandbox; no deployment into myscoutee-backend

Start with the helper

```
./setup-vagrant.sh           # user-local Vagrant + QEMU provider
./lab.sh up kvm ansible      # QEMU/KVM, no system libvirt
./lab.sh verify
./lab.sh apply puppet       # converge the same VM with Puppet
```

The host entry point is http://127.0.0.1:18080/. The lab uses the application's /nginx-health response shape as its concrete configuration exercise. It is a stand-alone teaching edge, not the application's production nginx image.

This guide explains the components, the method, and the terminal output. Follow the same experiment with either tool and compare the evidence.

The page at port 18080 shows the configured result. Ansible and Puppet are controlled here through terminal output from ansible-playbook and puppet apply; this lab has no management web console. Separate web consoles exist in Ansible AWX and Puppet Enterprise.

1. Basic ideas

Configuration management describes how a machine should look, then makes its actual state match. A successful run needs two kinds of evidence: the tool reports what it changed, and an independent check shows the service behaves as expected.

Term	Meaning in this lab
Host / guest	The host is your computer. The guest is the disposable Ubuntu VM whose nginx configuration is managed.
Desired state	nginx is installed and running; the shared site and page have the specified content; the active-tool marker has the selected value.
Actual state	The packages, files, service and HTTP response currently present inside the VM.
Convergence	A run compares actual state with desired state and fixes differences. The first run usually changes several resources.
Idempotence	Running the same declaration again leaves already-correct resources unchanged. Ansible shows changed=0; Puppet prints no resource-change Notice.
Drift	A manual or external change that moves the VM away from the declared state, such as replacing the managed HTML page with 'drift'.
Control / target	The control side starts the tool. The target is the VM being changed. In this small lab both tools execute inside the target VM.

The browser page is an output to inspect. The tool interface for this lab is the terminal plus the declarations in the repository. Ansible AWX and Puppet Enterprise are separate web products.

2. Components and data flow

```
host browser/curl :18080
        | Vagrant loopback port forwarding
Ubuntu guest :8080 -> nginx -> static lab page / status
        ^
        | Ansible or Puppet -> same shared files and service
```

Component	Job	Where
Vagrant + provider	Creates the VM with QEMU/KVM, optional libvirt or VMware, syncs source and forwards host port 18080 to guest port 8080.	Vagrantfile
Command wrapper	Selects a provider and a tool; calls Vagrant up, provision, verify or halt.	lab.sh
Bootstrap	Installs Ansible, Puppet and curl. Puppet's background agent is disabled for a controlled comparison.	provision/bootstrap.sh
Ansible	A playbook runs ordered tasks, using modules such as apt, file, copy and service. Here it connects locally inside the VM.	ansible/site.yml
Puppet	A manifest declares package, file and service resources. puppet apply builds and applies a catalog locally; no Puppet Server is used.	puppet/site.pp
Shared input	The nginx site, HTML page and synthetic JSON are identical for both tools.	shared/
nginx + verifier	nginx serves the resulting files; curl and the marker check what actually happened.	guest :8080 / lab.sh verify

Why this MyScoutee use case?

The application gateway already exposes /nginx-health with UP. The lab reproduces that small operational contract, then shows how configuration management keeps an edge host converged. The /lab/status.json endpoint explicitly says its backend is synthetic. It cannot be used to assess real application readiness.

A single source for both tools

Ansible and Puppet read the same files under shared/; the apply command syncs them into the VM first. Their declarations differ, but the HTTP result should match. Only one tool runs at a time; /etc/myscoutee-lab/tool records the last one.

3. Vagrant and Terraform

Both can own the VM lifecycle. Neither is Ansible or Puppet: those two tools configure the guest operating system after it exists. The choice of VM tool does not change the nginx desired state.

Question	Vagrant in this repository	Terraform alternative
What is declared?	A disposable local Ubuntu box, provider, memory, forwarded port and provisioners in Vagrantfile.	Libvirt resources such as VM, volumes and network in .tf files; Terraform tracks their identities in state.
How is it started?	./lab.sh up kvm ansible calls vagrant up with the QEMU provider using KVM acceleration.	terraform plan previews the resource changes; terraform apply creates them.
How does Ansible run?	Vagrant calls provision/configure.sh inside the VM; it invokes ansible-playbook with localhost inventory.	Terraform would create the VM; an explicit SSH/Ansible step or wrapper would then configure it. This repo has no Terraform lab path.
Local conveniences	Vagrant configures the rsync folder, SSH access and host port forwarding for this lab.	Those connections and source transfer would need explicit Terraform/provider and wrapper configuration.
When useful?	Quick, disposable local comparison on QEMU/KVM, optional libvirt or VMware Workstation/Fusion.	When the VM, storage and network should follow an infrastructure plan/state workflow.

The existing MyScoutee Terraform configuration in myscoutee-microservices/infra/terraform creates a larger, k3s-specific VM. It is a reference for libvirt patterns, not a dependency of this sandbox. No Ansible or Puppet change here is applied to myscoutee-backend.

Exact chain used by this repository

```
./lab.sh up kvm ansible
  -> vagrant up --provider=qemu (KVM acceleration)
  -> provision/bootstrap.sh (install tools)
  -> provision/configure.sh
  -> ansible-playbook -i 'localhost,' -c local /vagrant/ansible/site.yml
```

For Puppet, the same chain ends with puppet apply /vagrant/puppet/site.pp. The earlier direct-QEMU test ran these same guest scripts before Vagrant was installed. Terraform remains outside this sandbox.

4. A repeatable method

Use the same sequence each time. It separates what the tool says from what the service does, so a green process exit alone is never the entire conclusion.

Step	Action	Evidence to read
1. Define	Read the playbook, manifest and shared files before running.	Know the expected package, files, service, marker and HTTP response.
2. Apply	Run Ansible once on the VM.	Which tasks say changed? Did any task fail?
3. Verify	Check the marker and call the HTTP endpoints.	Marker says ansible; /nginx-health says UP; status says synthetic/ready.
4. Repeat	Run the same Ansible playbook again.	changed=0, no failures: the desired state is stable.
5. Compare	Apply Puppet twice on the same VM.	First run may switch the marker; second has no resource-change Notice.
6. Disturb	Replace the managed HTML page with 'drift'.	The browser now shows drift, proving the observed state changed.
7. Repair	Apply the selected tool and verify again.	It reports the page correction; the original page returns.

This method tests convergence (a correction happens), idempotence (the next run makes no correction), and observable behavior (nginx serves the expected response). The active-tool marker changes when switching tools; that single change is expected even if the nginx content is already identical.

5. Read the Ansible terminal

Ansible runs a playbook against an inventory. Here the inventory is localhost and the connection is local, so the playbook runs inside the VM. Each named task calls a module to check and, if needed, change one part of the guest.

```
$ ./lab.sh up kvm-local ansible
TASK [Install shared lab page] .......... ok: [localhost]
TASK [Record active config tool] ....... changed: [localhost]
PLAY RECAP
localhost : ok=9 changed=1 unreachable=0 failed=0
```

These lines were observed through Vagrant's QEMU/KVM provider on the local Ubuntu 24.04 box. Only the tool marker needed changing because the previous demo had already configured nginx. A fresh Bento box will have more changes on its first run.

Output	How to read it
ok: [localhost]	The task completed and found the resource already correct.
changed: [localhost]	The task made a correction. Read the task name to see what was touched.
ok=9	Nine tasks completed. This includes tasks that reported changed.
changed=1	One task modified the VM: here, the active-tool marker.
unreachable=0 / failed=0	The target was reachable and no task failed. This is necessary but does not replace HTTP verification.

```
$ ./lab.sh apply ansible
PLAY RECAP
localhost : ok=9 changed=0 unreachable=0 failed=0
```

The repeat has changed=0. The playbook is idempotent for this state. The command reports configuration work; use ./lab.sh verify to check the served result.

6. Read the Puppet terminal

A Puppet manifest declares resources: a package, directories, files and a service. puppet apply compiles those declarations into a catalog and applies it locally. This lab has no Puppet Server, agent polling cycle or Puppet web console.

```
$ ./lab.sh apply puppet
Notice: Compiled catalog for myscoutee-config-lab.
Notice: File[/etc/myscoutee-lab/tool]/content: content changed
Notice: Applied catalog in 0.30 seconds
```

These lines were observed through Vagrant on the local box. The catalog compiled, then Puppet changed the marker from ansible to puppet. Puppet identifies the exact resource path. Elapsed time varies between runs.

```
$ ./lab.sh apply puppet
Notice: Compiled catalog for myscoutee-config-lab.
Notice: Applied catalog in 0.19 seconds
```

On the repeat, there is no resource-change Notice. That is Puppet's normal quiet output for already-correct resources; unlike Ansible, this command does not print a per-task recap by default.

Read drift repair

```
$ ./lab.sh ssh -c 'sudo sh -c "echo drift > /srv/myscoutee-lab/index.html"'
$ ./lab.sh apply puppet
Notice: File[/srv/myscoutee-lab/index.html]/content: content changed
```

The file Notice means Puppet restored the declared HTML. Confirm with ./lab.sh verify or by opening the host page. The lab may also print a Ruby/Augeas warning on Ubuntu 24.04; the run above still completed and the service check passed.

7. Runbook and exercises

Before starting

Run ./setup-vagrant.sh to install Vagrant 2.4.9 and the QEMU provider in your user directory. The default kvm path needs QEMU and access to /dev/kvm, without system libvirt. Optional libvirt and vmware paths need their own provider plugins and host services. Reserve about 2 GiB RAM, 15 GiB disk and free port 127.0.0.1:18080; install rsync and curl. On a fresh host, the regular kvm path downloads roughly 2 GiB for the pinned Bento 202502.21.0 box, plus Ubuntu packages. Later starts use the cached box. QEMU reads its libvirt-format disk.

```
./setup-vagrant.sh
./lab.sh up kvm ansible
./lab.sh verify
curl -fsS http://127.0.0.1:18080/nginx-health
curl -fsS http://127.0.0.1:18080/lab/status.json
```

Only on the machine with the earlier Ubuntu 24.04 demo disk, run ./setup-local-box.sh after stopping that demo VM, then ./lab.sh up kvm-local ansible. This optional local path avoids a second box download; the normal kvm command is the portable default.

Compare convergence

```
./lab.sh apply ansible
./lab.sh apply ansible      # expect zero managed changes
./lab.sh apply puppet
./lab.sh apply puppet       # expect no new changes
./lab.sh verify
```

Repair a deliberate drift

```
./lab.sh ssh -c 'sudo sh -c "echo drift > /srv/myscoutee-lab/index.html"'
./lab.sh apply puppet       # or ansible, whichever is active
./lab.sh verify
```

The selected tool should restore the page. Reapplying a second time should leave the managed resources unchanged. Switching tools only changes the marker when the shared content is already identical.

Stop or remove

```
./lab.sh status
./lab.sh halt
./lab.sh destroy     # deletes this lab VM and its guest data
```

8. Validation and boundaries

Check	Expected result
GET /nginx-health	HTTP 200 with body UP followed by a newline
GET /lab/status.json	Synthetic JSON with status ready and backend synthetic
GET /	MyScoutee configuration lab page
Tool marker	Exactly ansible or puppet, matching the last apply
Second same-tool apply	No managed resource changes
After page drift + apply	Original HTML restored

When a check fails

Symptom	Inspect
No Vagrant command	Run ./setup-vagrant.sh; lab.sh finds its user-local binary.
QEMU/KVM unavailable	Check qemu-system-x86_64 and user access to /dev/kvm.
Optional libvirt unavailable	Install the libvirt plugin and start the system libvirt service.
VMware provider unavailable	Install the VMware provider plugin and utility; verify VMware itself works.
Guest provisioning failed	Read Vagrant output, then use ./lab.sh ssh to inspect apt or nginx -t.
Guest passes, host port fails	Check port 18080 availability and provider forwarding configuration.

Operating boundary

The lab manages only its disposable guest. It has no TLS, domain, secrets, backend proxy routes, production images, monitoring integration or persistent MyScoutee data. Its HTTP status is an exercise artifact. Do not point external traffic at it or treat it as a deployment recipe.

Source references

Backend health route: myscoutee-backend/config/nginx/http/spring.conf. Provider and forwarding semantics: developer.hashicorp.com/vagrant/docs/providers and developer.hashicorp.com/vagrant/docs/networking/forwarded_ports. Web consoles: docs.ansible.com/ecosystem.html and help.puppet.com/pe/current/topics/complete_install_log_in.htm.

Terminal excerpts on pages 6-7 were observed through Vagrant QEMU/KVM on the optional local Ubuntu 24.04 box on 2026-10-28. The portable path targets Ubuntu 22.04; versions and change counts can differ.